

# Lecture 1 - Perceptrons, linear separation, and XOR

ITC604 - use with the syllabus units I and II

## Learning goals for this lecture

- Write  $y = \text{step}(w \cdot x + b)$  and name every symbol.
- Give weights that realise AND and OR.
- Show that XOR is not linearly separable.
- Explain, in one paragraph, why a hidden layer fixes XOR.

## The biological sketch, then the model

A biological neuron collects incoming spikes on its dendrites, sums them near the soma, and fires an axon spike if the sum crosses a threshold. The perceptron is a cartoon of that: real-valued inputs, a weighted sum, a threshold. It is not a model of cortex. It is a linear classifier that we can train.

## Forward pass

Let  $x$  be an  $n$ -dimensional vector. Let  $w$  be the same length. The pre-activation is  $z = w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b$ . The output is  $y_{\text{hat}} = 1$  if  $z \geq 0$ , else 0. Equivalently  $y_{\text{hat}} = \text{step}(z)$ . The set of points with  $z = 0$  is a hyperplane. Changing  $w$  rotates the hyperplane; changing  $b$  shifts it.

## Boolean functions

AND: weights (1, 1), bias -1.5. Both inputs must be 1 before  $z$  is positive. OR: weights (1, 1), bias -0.5. Either input is enough. NAND is AND with the sign of the weights and bias flipped. XOR cannot be written this way. Plot the four points and try: any line that puts (0,1) and (1,0) on the positive side puts at least one of (0,0) or (1,1) there too.

## Perceptron training rule

Present examples one at a time. If the prediction matches the label, do nothing. If the true label is 1 and we predicted 0, add  $\eta \cdot x$  to  $w$  and  $\eta$  to  $b$ . If the true label is 0 and we predicted 1, subtract. This is mistake-driven learning. If a separating hyperplane exists, the algorithm finds one in a finite number of mistakes. That is the perceptron convergence theorem. If the data is not linearly separable, the updates never settle.

## From one layer to two

Hidden unit  $h_1$  computes  $x_1$  AND NOT  $x_2$ . Hidden unit  $h_2$  computes NOT  $x_1$  AND  $x_2$ . The output unit ORs  $h_1$  and  $h_2$ . That network is XOR. In the next lecture we replace the step with a sigmoid or ReLU so we can take gradients and learn those hidden features instead of designing them by hand.

## What to read before Lab 1

Implement the perceptron in about twenty lines of Python. Use  $\eta = 0.1$ . Draw the line  $w_1 x + w_2 y + b = 0$  after every epoch. The handout [perceptron-learning-handout.pdf](#) works the same examples with numbers.